

■ Metadata Dashboard - Documentation and Reflection

1. Documentation of Work

I built a Streamlit dashboard to explore the COVID-19 metadata dataset (metadata.csv with ~821,005 rows). Since the dataset was too large to handle in one go, I split the file into smaller parts (metadata_part_0.csv, metadata_part_1.csv, ...). The app loads all parts, merges them, and then allows the user to: - Select a column from the dataset. - View value counts (distribution) of the selected column. - Visualize the results as a bar chart. Lightweight mode was introduced to load only essential columns (e.g., title, license, abstract, publish_time, authors, journal) to improve performance.

2. Code with Comments (Excerpt from app.py)

```
import streamlit as st import pandas as pd import os st.title("■ Metadata Dashboard (Lightweight Mode)") # Define path to dataset parts data_folder = "C:/Users/user/Documents/metadata_parts" # Function to load and merge data parts @st.cache_data def load_metadata(): all_parts = [] for file in sorted(os.listdir(data_folder)): if file.endswith(".csv"): st.write(f>Loading {file} ...") # Show progress df = pd.read_csv(os.path.join(data_folder, file), usecols=["title", "license", "abstract", "publish_time", "authors", "journal"]) all_parts.append(df) return pd.concat(all_parts, ignore_index=True) # Load dataset df = load_metadata() st.write(f"Number of rows loaded: {len(df)}") # Show available columns st.write("Columns:", list(df.columns)) # Dropdown to select a column col = st.selectbox("Select a column to plot", df.columns) # Value counts if col: st.write(f"Value counts for {col}") counts = df[col].value_counts().head(20) st.bar_chart(counts)
```

3. Brief Report of Findings

- The dataset is very large, which made it difficult to process directly in memory. Splitting into parts solved this issue. - Columns such as 'title' and 'abstract' are very text-heavy, which increases load time compared to categorical columns like 'journal' or 'license'. - The dashboard makes it easy to quickly explore which values dominate certain columns (e.g., most frequent journals or licenses).

4. Reflection: Challenges and Learning

Challenges: - Handling a very large dataset (~821k rows) caused performance and memory issues. - Initial errors came from trying to run the app from the wrong directory in PowerShell/Command Prompt. - Some columns showed 'loading' for long periods, which was due to text-heavy fields. What I Learned: - Importance of data preprocessing (splitting files, selecting relevant columns). - How to use Streamlit for rapid dashboard development. - Practical troubleshooting in VS Code/PowerShell (navigating directories, running Python commands). - Optimization techniques like caching and column selection to improve speed. Next Steps / Improvements: - Add search/filter functionality for abstracts and authors. - Implement pagination for viewing large text fields. - Deploy the dashboard online (Streamlit Cloud or Heroku).